

Simple Control $P_{\text{Fuzzy}}+I$

Por: Sam Clark – 2025

Ref: <https://www.mathworks.com/matlabcentral/answers/2175759-pid-tuning-using-a-fuzzy-logic-controller>

1. Introdução

Um exemplo envolvendo uma planta de primeira ordem, onde um controlador P+I simples é projetado.

Nesse caso, apenas o ganho proporcional é descrito pelo sistema Fuzzy, tornando-o um sistema SISO (entrada única, saída única). A ação de controle integral permanece linear.

Foi utilizado o sistema Fuzzy sugeno para facilitar o design, definindo 3 funções de pertinência (membership functions - MFS) de entrada e 2 singletons de saída.

A principal diferença entre Mamdani e Sugeno é que as funções de pertinência de saída de Sugeno são lineares ou constantes.

Um singleton, ou mais precisamente um Fuzzy Singleton, é um conjunto difuso com uma função de associação que é unidade em um único ponto particular no universo do discurso e zero em qualquer outro lugar.

As etapas de design requerem conhecimento do controlador PI automático e a resposta do passo do sistema de circuito fechado.

Para iniciantes, recomenda-se seguir a abordagem à seguir para projetar um controlador Fuzzy que supere o controlador PID linear. Também, para iniciantes, quanto menor o número de MFs e regras, mais fácil o design de controle difuso se torna.

2. Código

```
% Controle Fuzzy P + I
% Por: Sam Clark - 2025
% Ref: https://www.mathworks.com/matlabcentral/answers/2175759-pid-tuning-using-a-fuzzy-logic-controller
% Adaptado por: Dr. Arnaldo de Carvalho Junior - 2025
```

```
% Step 1: obtain the Plant, Gp
```

```
Gp = tf(1, [1 1])
```

```
% Step 2: let the Autotuner find the PI gains
```

```
Gc = pidtune(Gp, 'PI')
```

```
Kp = Gc.Kp; % proportional gain
```

```
Ki = Gc.Ki; % integral gain
```

```
% Step 3: find the max amplitude of step response of closed-loop system
```

```
Gcl = minreal(feedback(Gc*Gp, 1));
```

```
tspan = linspace(0, 10, 101);
```

```
[y1, t1] = step(Gp, tspan); % generate step response of Gp
```

```
[y2, t2] = step(Gcl, tspan); % generate step response of Gcl
```

```
figure(1)
```

```
step(Gcl) % to check the maximum amplitude of the response under linear PI
```

```
controller
```

```
% Step 4: design Fuzzy P Controller
```

```
fis = sugfis;
```

```
% step 4a: describe the Fuzzy Input
```

```
fis = addInput(fis, [-1.5 +1.5], 'Name', 'Err'); % step response is under 1.2, so  $\pm 1.5$  is more than enough
```

```
fis = addMF(fis, 'Err', 'linzmf', [-0.75, 0.00], 'Name', 'N'); % symmetrical design (designer's choice)
```

```
fis = addMF(fis, 'Err', 'trimf', [-0.75 0 0.75], 'Name', 'Z'); % symmetrical design (designer's choice)
```

```
fis = addMF(fis, 'Err', 'linsmf', [ 0.00, 0.75], 'Name', 'P'); % symmetrical design (designer's choice)
```

```
% step 4b: describe the Fuzzy Output
```

```
lv = Kp-0.2; % low value of Kp (designer's choice)
```

```
hv = Kp+2.2; % high value of Kp (designer's choice)
```

```
fis = addOutput(fis, [1 3], 'Name', 'U'); % setting the range in SugFIS has no effect!
```

```
fis = addMF(fis, 'U', 'constant', lv, 'Name', 'L'); % 'constant' is singleton MF
```

```
fis = addMF(fis, 'U', 'constant', hv, 'Name', 'H');
```

```
% step 4c: describe the Fuzzy Rules
```

```
rules = [
```

```
"Err==N => U=L" % follow this rule (NOT designer's choice!)
```

```
"Err==Z => U=H" % follow this rule (NOT designer's choice!)
```

```
"Err==P => U=L" % follow this rule (NOT designer's choice!)
```

```
];
```

```
fis = addRule(fis, rules);
```

```
figure(2)
```

```
subplot(211)
```

```
plotmf(fis, 'input', 1), grid on,
```

```
xlabel('Error')
```

```

title('Membership functions of Error input')
grid on;

subplot(212)
opt = gensurfOptions('NumGridPoints', 301); % generate 301 points in the interval
[-1.5, +1.5]
gensurf(fis, opt)
xlabel('Error'), ylabel('Kp')

title ('Fuzzy control gain, Kp')

grid on;

% Step 5a: write the code for Control system (Plant and Controller)
function dx = plant(t, x, fis, Ki)
% definitions
r = 1; % reference
e = r - x(1); % error

% fuzzy controller
fKp = evalfis(fis, e); % fuzzy Kp only
u = fKp*e + Ki*x(2); % PI controller

% differential equations
dx(1) = - 1*x(1) + 1*u; % plant state space = tf(1, [1 1])
dx(2) = e; % x2 is the integral of error
dx = [dx(1)
dx(2)];
end

% Step 5b: call ode45 to solve differential equations in the Control System
x0 = [0; 0]; % initial condition
[t, x] = ode45(@(t, x) plant(t, x, fis, Ki), tspan, x0);
figure(3)
hold on
plot(t, x(:,1)), grid on % plot step response of plant under fuzzy P+I controller
plot(t2, y2), % plot step response of plant under PI controller
plot(t1, y1), % plot step response of original plant
hold off
title('Step responses'), xlabel('Time / s'), ylabel('Amplitude')

legend('Fuzzy P+I Ctrl', 'PI Controller', 'Original Plant', 'location', 'east')

grid on;

```